
PROGRAMMATION ORIENTÉE OBJET IG2I, LE2 – 2025-2026

TP6 – EXCEPTIONS, ENTRÉES/SORTIES

Nathan Davouse, Gaël Guillot, Maxime Ogier

Contexte et objectif du TP

Ah le nord ! Partie deux ! On disait qu’au nord ils boivent beaucoup de bière. Bon, toutes les caisses sont encore au dépôt. Elles attendent vos indications pour être livrées.

Dans ce TP, nous allons d’abord étudier le concept d’exception. Nous allons ensuite mettre en œuvre des méthodes qui nous permettront de lire les fichiers de données proposés par Monsieur Houblon : chaque jour il vous donnera un fichier texte avec les coordonnées des clients à livrer ainsi que leur demande. Nous aimons bien bosser, mais nous ne voulons pas rentrer à la main, chaque matin, la liste des clients à livrer : on préfère tout de même le faire d’une manière automatisée.

Enfin, nous nous intéresserons à optimiser le planning de livraison pour faire faire des économies à Monsieur Houblon. Si nous lui proposons un planning moins cher, le gain se répercutera sur le prix de la bière même, et tout le monde sera gagnant !

Ce TP permet d’illustrer les notions suivantes :

- la notion d’exception ;
- les blocs **try** et **catch** pour la gestion des exceptions.
- la lecture d’un fichier de texte et l’écriture dans un fichier de texte à l’aide des classes **PrintWriter** et **FileReader** ;
- utilisation des collections en Java ;
- des notions d’algorithmique.

Bonnes pratiques à adopter

- Testez votre code au fur et à mesure de votre avancement.
- Respectez les conventions de nommage Java.
- Faites des indentations de manière correcte.

- Donnez des noms compréhensibles et pertinents à vos variables, attributs, classes, méthodes.
- Respectez autant que possible le principe d’encapsulation.
- Utilisez des structures de données adaptées.
- Faites des méthodes courtes avec peu de niveaux d’indentation.
- Ne faites pas de duplication de code, mais privilégiez le code réutilisable.
- Utilisez les outils de votre IDE pour faire du *refactoring* (clic-droit puis ‘*Refactor*’ sur IntelliJ IDEA).
- Utilisez la Javadoc pour comprendre le fonctionnement des méthodes que vous appelez.
- Quand c’est nécessaire, commentez votre propre code de façon pertinente au format Javadoc (commentaire commençant par “/” au-dessus des définitions des attributs et méthodes).

1 Exceptions

1.1 Théorie : lever et gérer des exceptions

Même lorsqu’un programme est au point, certaines circonstances *exceptionnelles* peuvent compromettre la poursuite de son exécution ; il peut s’agir par exemple de données incorrectes rentrées par l’utilisateur, ou encore de la rencontre d’une fin de fichier prématurée (alors qu’on a besoin d’informations supplémentaires pour continuer le traitement).

On pourrait toujours essayer d’examiner toutes les situations possibles au sein du programme (avec plein de conditions **if**, **else**, **switch**, **case**, ...) et donc prendre les décisions qui s’imposent chaque fois qu’une circonstance exceptionnelle survient.

Cependant, cette approche pose deux problèmes : (1) le programmeur risque d’omettre certaines situations ; et (2) les codes risquent d’être très difficiles à lire.

Java dispose d’un mécanisme très souple nommé *gestion d’exceptions*, qui permet à la fois : (1) de dissocier la détection d’une anomalie de son traitement ; (2) de séparer la gestion des anomalies du reste du code, donc de contribuer à la lisibilité des programmes.

En général, (1) les exceptions sont déclenchées par l’instruction **throw**, et (2) les instructions qui peuvent les déclencher sont mises dans un bloc appelé **try** et elles sont récupérées/traitées dans un bloc appelé **catch**.

Nous allons à présent nous familiariser avec ces instructions dans un premier exemple.

1.1.1 La classe `Exception`

Le langage Java dispose d'une classe **Exception** qui est la classe mère de tous les types d'exception qui peuvent survenir dans un programme. Cette classe permet de garder en mémoire des informations comme la pile des appels qui a mené au déclenchement de l'exception, ou bien des détails sur l'exception déclenchée. Remarquez que certains types d'exceptions existent déjà dans le langage Java (vous avez déjà dû voir **NullPointerException** par exemple). Mais il est également possible de créer notre propre type d'exception. Pour cela, il suffit de créer une classe qui hérite de **Exception**.

1.1.2 Lever une exception

Dans le corps d'une méthode (ou d'un constructeur), si l'on remarque une anomalie, au lieu de chercher un traitement par défaut, il est possible de déclencher une exception en utilisant l'instruction **throw** suivie d'un objet de type **Exception** (ou d'une classe qui hérite de **Exception**). L'appel à cette instruction arrête alors le déroulement de la méthode et renvoie directement l'objet de l'instruction **throw** au lieu de renvoyer un objet du type de retour de la méthode. Par ailleurs, dans ce cas, il est nécessaire d'indiquer dans la signature de la méthode qu'elle peut potentiellement renvoyer un certain type d'exception. On utilise pour cela le mot clé **throws** (faites bien attention au 's' final).

Voici un exemple de code dans lequel la méthode **methodeAvecException** peut lever une exception de type **ExceptionType** :

```
1 public int methodeAvecException(double val) throws ExceptionType {
2     if(val < 0) { // test sur la valeur de val
3         ExceptionType ex = new ExceptionType();
4         throw ex;
5     } else {
6         // déroulement normal de la methode
7         // ...
8         return ...; // retourne une valeur entiere
9     }
10 }
```

1.1.3 Gestion des exceptions

Voyons maintenant comment procéder pour gérer convenablement les éventuelles exceptions qui peuvent être déclenchées lors de l'appel d'une méthode. Tout d'abord, le langage Java ne permet pas d'appeler une méthode qui déclare dans sa signature qu'elle peut lever une exception (avec le mot clé **throws**) sans faire un "traitement particulier". Pour que le code compile, il faut choisir parmi deux possibilités, comme présenté par la suite.

1. La méthode appelante déclare elle-même qu'elle peut potentiellement déclencher une exception. Par exemple, on pourrait avoir le code suivant :

```
1 public void methodeAppelante() throws ExceptionType {
2     // ... debut de la methode
3
4     double val = ...;
5     int res = methodeAvecException(val);
6
7     // ... fin de la methode
8 }
```

2. La méthode appelante fait l'appel à une méthode qui peut déclencher une exception au sein d'un bloc **try**. Ce bloc **try** est immédiatement suivi d'un bloc **catch**, appelé *gestionnaire d'exception* dans lequel on définit ce qu'il se passe si une exception a été déclenchée. Ce gestionnaire d'exception prend en paramètre un objet du type d'exception qui peut être déclenchée dans le bloc **try**. Par exemple, on pourrait avoir le code suivant :

```
1 public void autreMethodeAppelante() {
2     // ... debut de la methode
3
4     try {
5         // ...
6         double val = ...;
7         int res = methodeAvecException(val);
8         // ...
9     } catch (ExceptionType ex) {
10        // si une exception est declenchee a la ligne 7
11        // on arrive directement dans ce bloc catch
12        // on indique ce qu'on fait dans ce cas la
13    }
14
15    // ... fin de la methode
16 }
```

Par ailleurs, notez bien que si on choisit la première option, il faudra ensuite se poser la même question pour le traitement de l'exception déclenchée par la méthode appelante. Donc à un certain moment, il faudra bien gérer l'exception dans un gestionnaire d'exceptions. De plus, tout l'intérêt de ce mécanisme de gestion des exceptions est sa souplesse. La méthode **methodeAvecException** peut être appelée à plusieurs endroits dans le code, et à chaque fois on pourrait utiliser un gestionnaire d'exception différent, dans lequel on ferait un traitement

différent, même si c'est la même exception qui a été déclenchée. Ainsi, on peut vouloir à certains endroits afficher un message avant d'arrêter l'exécution du programme, ou bien ne rien afficher, ou encore tenter de trouver une solution par défaut. Ceci permet donc d'apporter beaucoup de souplesse à la gestion des exceptions dans votre code. Imaginez à quoi ressemblerait la méthode **methodeAvecException** si elle ne déclenchait pas l'exception, et qu'elle devait la gérer avec différents cas.

Lors de l'utilisation des blocs **try** et **catch**, notez que le bloc **catch** possède un paramètre qui correspond à l'objet fourni par l'instruction **throw** dans la méthode appelée dans le bloc **try**. Le gestionnaire d'exception peut donc utiliser cet objet pour accéder à certaines informations sur l'exception. Pour ce faire, il suffit de fournir les attributs ou méthodes appropriés dans la classe d'exception correspondante.

Enfin, remarquez que si aucune exception n'a été levée lors de l'exécution du bloc **try**, le bloc **catch** n'est pas exécuté et l'exécution du programme se poursuit avec les instructions qui suivent le bloc **catch**. Si une exception a été levée lors de l'exécution du bloc **try**, le bloc **catch** est exécuté immédiatement et l'exécution du programme se poursuit avec les instructions qui suivent le bloc **catch**. Cela signifie que si le bloc **try** contient des instructions après l'appel à une méthode qui peut lever une exception, ces instructions ne seront pas exécutées si une exception est levée.

1.1.4 Exemple d'exceptions dans le langage Java

Une des exceptions standards du langage Java et que vous avez certainement déjà rencontrée est l'exception **NullPointerException** lorsque par exemple vous tentez d'accéder à un attribut ou de faire appel à une méthode d'un objet non initialisé (qui vaut donc **null**).

Une autre exception standard du langage Java est l'exception **IOException** utilisée par les méthodes d'entrées/sorties. Elle est utilisée pour signaler des interruptions ou des échecs des opérations d'*input* ou d'*output* (I/O).

1.2 Exception dans le constructeur de la classe Client

Dans le projet **Livraison** commencé au TP précédent, considérons la classe **Client**. Nous savons que nous ne pouvons pas livrer à un client un nombre de caisses négatif. En effet, cela n'a pas de sens, et si nous appelons notre algorithme de construction de tournées avec des demandes négatives, les résultats aussi n'auront aucun sens ! De plus, pour livrer un client il faut lui apporter au moins une caisse de bière. On pourrait envisager de faire un traitement par défaut (comme nous l'avons déjà fait précédemment dans d'autres TPs). Mais le désavantage de ces traitements par défaut est que l'utilisateur ne se rend pas compte qu'il y a un problème, et le programme continue son exécution comme si tout s'était passé normalement. Il n'est pas toujours souhaitable d'avoir ce type de comportement.

Nous proposons ici, au sein du constructeur de la classe **Client**, de vérifier la validité des paramètres fournis et lorsque la quantité à livrer au client est incorrecte nous déclenchons alors une exception.

Question 1. Après avoir ouvert IntelliJ IDEA, reprenez le code développé lors de la dernière séance. Créez un nouveau paquetage **exceptions** qui contiendra vos types d'exceptions. Ajoutez une classe **ExceptionQuantite** qui représente l'exception que l'on vient de décrire. Cette classe héritera de la classe **Exception**. Pour le moment elle est vide, nous reviendrons la compléter plus tard.

Question 2. Modifiez le constructeur de la classe **Client** de façon à déclencher une exception de type **ExceptionQuantite** si la quantité à livrer passée en paramètre est strictement inférieure à une caisse de bière.



Vous allez probablement voir des erreurs apparaître dans votre code. Ne vous inquiétez pas, c'est normal ! On réglera cela dans quelques instants.

Question 3. Ajoutez une méthode principale (si vous ne l'avez pas déjà) dans la classe **Client**. Écrivez-y puis exécutez le code suivant :

```
1 try {
2     Client c1 = new Client(5, 5, 10);
3     System.out.println("Creation ok");
4     Client c2 = new Client(5, -5, 0);
5     System.out.println("Creation ok");
6 } catch (ExceptionQuantite ex) {
7     System.out.println("Erreur: quantité negative");
8     System.exit(-1);
9 }
```

Que se passe-t-il ?



L'appel à la méthode **exit()** termine l'exécution du programme. Par convention, une valeur non nulle est passée à la méthode **exit()** si une interruption anormale du code s'est produite.

Question 4. Corrigez votre code (dans toutes les classes où de nouveaux clients sont créés) en gérant les exceptions déclenchées par votre nouveau constructeur (attention : effacer les appels au constructeur élimine les erreurs, mais ce n'est pas la bonne réponse).

L'exemple que vous venez de faire était illustratif, mais restrictif pour certaines raisons :

- le gestionnaire d'exception (le **catch**) ne reçoit aucune information ; il reçoit juste un objet qu'il n'utilise pas ; nous allons voir comment utiliser cet objet pour communiquer des informations au gestionnaire ;
- le gestionnaire d'exception se contentait d'interrompre le programme (appel à **exit()**) ; nous verrons qu'il est possible de poursuivre l'exécution.

Question 5. Ajoutez dans la classe **ExceptionQuantite** un attribut de type entier nommé **quantite**. Ajoutez un constructeur de la classe **ExceptionQuantite** par donnée de la quantité. Ajoutez les accesseurs et les mutateurs nécessaires et redéfinissez la méthode **toString()**. Mettez également à jour les appels au constructeur de **ExceptionQuantite** dans le reste du code.

Question 6. Dans la méthode principale de la classe **Client**, modifiez le gestionnaire d'exceptions pour imprimer des informations supplémentaires sur la raison de l'exception, notamment la valeur exacte de la quantité qui a provoqué le déclenchement de l'exception. Testez votre code.

La classe **Exception** (dont hérite votre classe) dispose d'un constructeur qui prend en argument un objet de type **String** qui représente un message lié à l'exception. La classe **Exception** dispose également d'une méthode **getMessage()** qui renvoie ce message. La classe **ExceptionQuantite** possède donc cette méthode **getMessage()** (car **ExceptionQuantite** hérite de **Exception**).

Question 7. Modifiez le constructeur de la classe **ExceptionQuantite** en ajoutant un argument de type **String** qui représente le message lié à l'exception. Veillez à appeler le bon constructeur de la classe mère **Exception**. Modifiez en conséquence les appels au constructeur de **ExceptionQuantite**.

Testez le bon fonctionnement de cette modification. Pour cela, dans le gestionnaire d'exception **catch**, vous pouvez imprimer le message de l'exception sur la console.

Question 8. Pour bien comprendre que nous pouvons poursuivre l'exécution de notre programme même après le déclenchement d'une exception ; dans la méthode principale de la classe **Client**, effacez l'appel à **System.exit (-1)**. Après le bloc **catch** ajoutez un appel à la méthode **System.out.println()** avec un message. Testez votre code. Retrouvez-vous ce dernier message dans la console ?

Dans notre exemple de livraison, on pourrait imaginer d'imprimer un message d'erreur lorsqu'un client ne demande pas au moins une caisse de bière. Par contre on peut imaginer de continuer l'exécution si le nombre de caisses demandées est zéro (auquel cas on peut tout de même afficher un message pour informer l'utilisateur).

Question 9. Modifiez le gestionnaire d'exception dans la méthode principale de la classe **Client** de telle sorte qu'un message soit affiché lorsqu'une exception de type **ExceptionQuantite** est déclenchée. L'exécution du programme continue si le client demande zéro caisse, et elle se termine si le nombre de caisses est négatif. Testez le bon fonctionnement de votre code.

Si l'appel au constructeur de **Client** lève une exception et que l'exécution du code se poursuit, le client a-t-il été créé ?

2 Planification de la livraison

Dans cette première partie, nous proposons de rajouter à notre modèle objet une classe **Routage** qui sera l'objet utilisé pour lire les données d'une instance du problème, résoudre le problème, et enfin écrire la solution dans un fichier. Notez que nous intéresserons à la lecture des données et l'écriture des solutions dans la section suivante.

Question 10. Créez un nouveau paquetage **planification** dans lequel on aura les objets qui permettent de fabriquer les plannings de la brasserie. Dans ce paquetage, créez une nouvelle classe **Routage**. Elle aura les attributs suivants : un attribut de type **Depot**, un ensemble de clients (**mesClients**) contenant les clients à livrer ; un attribut de type **Planning**, contenant le planning de la livraison ; et un entier représentant la capacité des véhicules (on suppose que cette capacité est identique pour tous les véhicules qui font les livraisons).

Ajoutez un constructeur par la donnée de la capacité des véhicules. Le constructeur initialise le dépôt au point de coordonnées $x = 0, y = 0$. Ajoutez un autre constructeur par la donnée d'un dépôt et de la capacité des véhicules.

Question 11. Ajoutez dans la classe **Routage** la méthode suivante qui permet d'initialiser l'ensemble des clients afin de réaliser un premier test :

```

1  private void creationClientsTest1() {
2      Client c0 = new Client(-99.7497, 12.7171, 4);
3      Client c1 = new Client(61.7481, 17.0019, 10);
4      Client c2 = new Client(-29.9417, 79.1925, 17);
5      Client c3 = new Client(49.321, -65.1784, 18);
6      Client c4 = new Client(42.1003, 2.70699, 7);
7      Client c5 = new Client(-97.0031, -81.7194, 8);
8      Client c6 = new Client(-70.5374, -66.8203, 20);
9      Client c7 = new Client(-10.8615, -76.1834, 1);
10     Client c8 = new Client(-98.2177, -24.424, 11);
11     Client c9 = new Client(14.2369, 20.3528, 13);
12     mesClients.clear();
13     mesClients.add(c0);
14     mesClients.add(c1);
15     mesClients.add(c2);
16     mesClients.add(c3);

```

```

17     mesClients.add(c4);
18     mesClients.add(c5);
19     mesClients.add(c6);
20     mesClients.add(c7);
21     mesClients.add(c8);
22     mesClients.add(c9);
23 }

```

Question 12. Écrivez une méthode **private void initialiserRoutes()** qui initialise les routes qui partent du dépôt et de chaque client. Rappelez-vous de la méthode **ajouterRoutes()** que vous avez codée dans la classe **Point**.

Question 13. Ajoutez dans la classe **Routage** une méthode **public void planificationBasique()**. Cette méthode doit faire appel à la méthode de planification basique de la classe **Planning**.

Question 14. Ajoutez une méthode statique **public static void test()**. Cette méthode crée un objet de type **Routage** (avec une capacité de 50 caisses de bière, valeur donnée par Monsieur Houblon). Puis, cette méthode fait appel à la méthode **creationClientsTest1()** puis à la méthode **initialiserRoutes()**. Enfin, on peut faire appel à la méthode **planificationBasique()** afin de planifier les tournées pour livrer les clients de l'ensemble de test. Puis vous afficherez les informations sur le nombre de tournées, la distance totale et la quantité livrée.

Question 15. Écrivez une méthode principale (**public static void main(String[] args)**) dans la classe **Routage** dans laquelle vous faites appel à la méthode statique **test()**. Quels résultats obtenez-vous en termes de nombre de tournées et de distance totale ? Le nombre de caisses livrées doit être de 109 (si vous n'en avez pas perdu en route ...).

3 Flux et fichiers de texte

Monsieur Houblon s'attend à avoir son planning de livraison bien imprimé dans un fichier : autrement il devrait se balader avec votre ordinateur pour examiner les informations imprimées dans votre console. De même, Monsieur Houblon vous donnera un fichier texte avec tous les clients à visiter. Il ne serait pas très efficace de devoir rentrer à la main tous les clients à livrer avec leurs coordonnées et leur demande.

3.1 Théorie : écriture et lecture dans des fichiers

Jusqu'à présent, nous avons affiché des informations dans la console en utilisant la méthode **System.out.println()**. En fait, **out** est ce que l'on nomme un *flux de sortie*. Dans le langage Java, la notion de flux est très générale puisqu'un flux de sortie désigne n'importe quel *canal* susceptible de recevoir de l'information

sous forme d'une suite d'octets (regroupement de 8 bits codant une information) : console, périphérique d'affichage, fichier, ou une connexion à un site distant.

De façon comparable, il existe des flux d'entrée, c'est-à-dire des canaux délivrant de l'information sous forme d'une suite d'octets. Là encore, il peut s'agir d'un périphérique de saisie (votre clavier), d'un fichier, d'une connexion ou d'un emplacement en mémoire centrale.

Le langage Java offre la possibilité d'automatiser l'écriture et la lecture des données sur des fichiers avec les classes **PrintWriter** et **FileReader**.

3.1.1 Écriture de texte dans un fichier

Pour écrire du texte dans un fichier, le langage Java fournit une classe **PrintWriter**. Cette classe est chargée d'écrire des données dans un fichier spécifié. La classe **PrintWriter** possède un constructeur par donnée du nom du fichier dans lequel on souhaite écrire. Par exemple, si votre fichier s'appelle **fichierSortie.txt**, l'initialisation d'un tel objet sera :

```

1  PrintWriter writer = new PrintWriter("fichierSortie.txt");

```

Si le fichier n'existe pas, il est créé automatiquement. Par contre, s'il existe, son contenu est effacé. La création d'un objet **PrintWriter** peut lever une exception de type **FileNotFoundException** si une erreur apparaît à la création ou à l'ouverture du fichier. Parmi les méthodes de la classe **PrintWriter**, nous avons la possibilité d'utiliser les méthodes **print()** et **println()**. Leur comportement est similaire à celui des méthodes **print()** et **println()** que vous utilisez pour afficher des informations sur la console. Par ailleurs, la classe **PrintWriter** dispose d'une méthode **close()** qui ferme le flux de sortie et libère les ressources système associées. Il ne faudra donc pas oublier de faire appel à cette méthode lorsque l'écriture est terminée.

Le texte que vous souhaitez écrire dans le fichier n'est pas directement écrit dans le fichier, mais il est d'abord stocké dans un tampon. Le tampon est vidé lors de l'appel à la méthode **flush()** ou lors de la fermeture du flux d'écriture avec l'appel à la méthode **close()**. Seulement à ce moment là, vous verrez apparaître le texte dans le fichier.

Pour résumer, l'écriture de texte dans un fichier peut se faire avec un code similaire au code suivant :

```

1  PrintWriter writer;
2  try {
3      writer = new PrintWriter("fichierSortie.txt");
4      writer.println("premiere ligne");
5      writer.flush(); // vide le tampon

```

```

6     writer.println("deuxieme ligne");
7     writer.close();
8 } catch (FileNotFoundException ex) {
9     // que fait-on si on n'a pas pu ouvrir le fichier
10 }

```

3.1.2 Lecture de texte depuis un fichier

Pour lire des informations depuis un fichier nous pouvons utiliser la classe **FileReader**. Un objet de cette classe est initialisé en lui passant en paramètre le nom du fichier à lire :

```

1   FileReader reader = new FileReader("fichierEntree.txt");

```

La classe **FileReader** ne permet que de faire de la lecture “de bas niveau”, c’est-à-dire qu’on ne peut que lire les caractères un par un, ainsi que savoir si la fin du fichier est atteinte. Ceci n’est pas très approprié pour lire un fichier avec plusieurs lignes. La classe **BufferedReader** prend en paramètre de son constructeur un objet de type **FileReader**, et permet ensuite de faire de la lecture ligne par ligne du fichier avec la méthode **readLine()**. Il reste alors juste à s’occuper de traiter les informations de chaque ligne.

Lors de la création d’un objet de type **FileReader**, une exception de type **FileNotFoundException** peut être levée. Cette exception est déclenchée lorsque le fichier que vous souhaitez ouvrir n’est pas trouvé. De plus, la méthode **readLine()** de la classe **BufferedReader** déclenche une exception de type **IOException**.

Comme pour l’objet **PrintWriter**, les objets de type **FileReader** et **BufferedReader** possèdent des méthodes **close()** qui permettent de fermer les flux d’entrée et de libérer toutes les ressources système associées. Il ne faut donc pas oublier de faire appel à ces méthodes lorsque la lecture du fichier est terminée.

Pour résumer, la lecture de texte dans un fichier peut se faire avec un code similaire au code suivant :

```

1   FileReader reader;
2   BufferedReader lineReader;
3   try {
4       reader = new FileReader("fichierEntree.txt");
5       lineReader = new BufferedReader(reader);
6       String ligne = lineReader.readLine();
7       while(ligne != null) {
8           // traitement de la ligne de texte

```

```

9       ligne = lineReader.readLine(); // lecture d’une nouvelle ligne
10  }
11  lineReader.close();
12  reader.close();
13 } catch (FileNotFoundException ex) {
14     // que fait-on si on n’a pas pu ouvrir le fichier
15 } catch (IOException ex) {
16     // que fait-on si en cas d’erreur d’entree/sortie
17 }

```

Remarquez dans le code précédent qu’il est possible de faire suivre un bloc **try** de plusieurs bloc **catch** qui définissent des gestionnaires d’exceptions différents en fonction du type d’exception qui est levé.

3.2 Écriture du planning de livraison dans un fichier

Le format texte du planning souhaité par Monsieur Houblon est le suivant. Dans la première ligne, on peut y lire les informations générales du planning : nombre de tournées, distance totale parcourue et nombre total de caisses livrées. Après, il souhaite avoir des informations sur chaque tournée. Pour chaque tournée, il souhaite avoir une première ligne explicative avec le nombre de clients visités, la longueur et le nombre de caisses livrées, ainsi que (sur les lignes suivantes) la liste des clients visités avec leurs coordonnées et la quantité demandée.

Question 16. Ajoutez dans la classe **Routage** une méthode **ecriturePlanning(String fichierSortie)**. Cette méthode doit écrire les informations relatives à la planification des livraisons dans le fichier nommé **outputFile**, conformément aux spécifications décrites ci-dessus.



Vous pouvez soit modifier directement les méthodes **toString** dans les différentes classes de manière à ce qu’elles permettent d’écrire les informations attendues, ou bien vous pouvez ajouter de nouvelles méthodes pour écrire spécifiquement la planification dans ce format. Dans les deux cas, vous devez déléguer le traitement dans les différentes classes et ne pas tout écrire dans une seule méthode d’une seule classe.

3.3 Lecture d’une instance dans un fichier

Dans cette section, nous voulons lire une instance, c’est-à-dire toutes les informations nécessaires pour résoudre le problème de routage, à partir d’un fichier. Dans ce cas, une instance consiste en : la capacité des véhicules, un dépôt, un ensemble de clients.

Question 17. Dans la classe **Routage**, ajoutez une méthode **lectureInstance(String fichierEntree)**. Pour l’instant, cette méthode doit simple-

ment lire chaque ligne du fichier nommé **fichierEntree**, et pour chaque ligne l'imprimer sur la console.

Dans le dossier de votre projet, ajoutez un fichier texte (**test.txt** par exemple) dans lequel vous avez écrit quelques lignes de texte. Pouvez-vous lire ce fichier ?

Monsieur Houblon vous dit qu'il vous donnera chaque jour un fichier texte dont la première ligne contient la capacité de ses véhicules (nombre de type **int**). Puis, il y aura une ligne pour chaque point du réseau routier utilisé pour la livraison. La première de ces lignes (donc la deuxième du fichier) contient deux nombres de type **double**. Ces nombres sont les coordonnées du dépôt. Les informations dans les autres lignes seront pour les clients (un client sur chaque ligne). Les deux premiers nombres (de type **double**) sont respectivement l'abscisse et l'ordonnée du client, le troisième (de type **int**) est la demande. Il y a une tabulation entre deux valeurs de chaque ligne.

Question 18. Modifiez la méthode **lectureInstance** pour qu'elle puisse traiter les données du fichier d'entrée et initialiser les attributs correspondants : capacité des véhicules, planification, dépôt, ensemble de clients. Testez votre méthode sur le fichier **testLecture0.txt** disponible sur Moodle.

Lorsque l'on modifie la capacité des véhicules, il faut veiller à modifier également le planning en créant un nouveau planning avec la bonne capacité de véhicules.



Lorsqu'une ligne contient plusieurs valeurs, vous pouvez utiliser la méthode **split(String val)** de la classe **String** pour diviser la chaîne en fonction du délimiteur **val**. Elle renvoie les différents morceaux de la chaîne dans un tableau de **String**.

Pour obtenir une valeur entière ou double à partir d'une chaîne, vous pouvez utiliser les méthodes statiques **parseInt** de la classe **Integer** et **parseDouble** de la classe **Double**.

Question 19. Lisez le fichier **testLecture1.txt** disponible sur Moodle. Trouvez-vous des erreurs de frappe de la part de Monsieur Houblon?

Question 20. Ajoutez dans la classe **Routage** une méthode statique **public static void testLecture(String nomFichier)**. Cette méthode crée un objet de type **Routage**, et lit le fichier **nomFichier** pour créer les clients à visiter et puis elle initialise les routes du réseau. Puis, vous pouvez faire appel à la méthode **planificationBasique()** afin de réaliser la planification. Enfin, écrivez la solution obtenue dans un fichier nommé **sol_** suivi de **nomFichier**.

Testez cette méthode avec les fichiers **test1.txt**, **test2.txt**, **test3.txt**, **test4.txt** et **test5.txt** disponibles sur Moodle.

4 Optimisation du planning

Nous avons mis tout en place pour pouvoir sortir des plannings de livraison sur les instances que Monsieur Houblon nous donnera. Par contre, l'algorithme qui calcule le planning n'est pas du tout optimisé. Nous voulons donc améliorer l'aspect algorithmique de notre outil.

4.1 Réduction du nombre de tournées

L'optimisation basique que nous avons écrit lors du dernier TP ajoute un client dans la tournée courante si la capacité résiduelle du véhicule est respectée. Sinon, une nouvelle tournée est créée pour accueillir le client. Nous notons ici que lorsque l'insertion d'un client dans une tournée échoue, la tournée n'est plus jamais prise en considération. Pourtant, le véhicule associé pourrait avoir de la place pour livrer d'autres clients (qui auraient une demande plus faible).

Question 21. Dans la classe **Planning**, ajoutez une méthode **planning-MinNbTours(Depot depot, Set<Client> clients)** qui itère sur l'ensemble des clients et qui, pour chacun d'entre eux, effectue les opérations suivantes. Tout d'abord, il tente d'insérer le client dans l'une des tournées existantes. Si aucune tournée ne dispose d'une capacité résiduelle suffisante pour ajouter le client, une nouvelle tournée est créée.

Testez ce nouvel algorithme et comparez les résultats avec ceux fournis par l'optimisation de base (en termes de nombre de tournées et de distance totale parcourue).



Notez qu'avec cet algorithme, les tournées pouvant être modifiées après avoir été ajoutées à la planification, il semble préférable de recalculer la distance totale parcourue de la planification à la fin de l'algorithme.

4.2 Meilleure insertion

Dans cette seconde section, nous allons voir l'algorithme de la meilleure insertion (*best insertion* en anglais). L'idée de cet algorithme est la suivante : étant donné un ensemble de tournées et un client, ce dernier est inséré dans la meilleure position possible dans les tournées existantes. Dans notre cas, la meilleure insertion est celle qui permet de réaliser le plus petit détour (tout en respectant la capacité du véhicule) pour livrer le client à ajouter. Pour ce faire, nous devons parcourir toutes les tournées de notre planning. Pour chaque tournée, on vérifie d'abord que l'insertion du client peut être réalisée, i.e. que la capacité du véhicule est respectée. Si tel est le cas, on cherche la position d'insertion du client dans la tournée qui permet de réaliser le plus petit détour possible. Ainsi, on n'insère plus le client en fin, mais on teste l'insertion du client juste après le dépôt et après chacun des clients déjà présents dans la tournée. La position qui permet de réaliser le plus petit détour est retenue. Jusque là on fait seulement des tests,

mais le client n'a été inséré dans aucune tournée. Le client est finalement inséré dans la tournée qui permet de réaliser le plus petit détour (et dans cette tournée, il est inséré à la position qui permet le plus petit détour).

Question 22. Créez dans le paquetage **planning** une classe **BestInsertion** avec au minimum les trois attributs suivant : un de type **Tournee**, un de type **double** et un de type **int** pour mémoriser respectivement la tournée, la longueur du détour et la position dans la tournée associée à la meilleure insertion. Ajoutez les constructeurs, accesseurs et mutateurs nécessaires. Il pourra être utile d'ajouter un constructeur par défaut (tournée **null** et distance du détour infinie).

Question 23. Ajoutez dans la classe **Tournee** une méthode **public BestInsertion calculerBestInsertion(Client nouveauClient)**. Cette méthode calcule la meilleure insertion du **nouveauClient** dans la tournée courante (sans réaliser cette insertion) et renvoie une instance de la classe **BestInsertion** avec les informations associées à la meilleure insertion. Si l'insertion n'est pas réalisable, il faudra renvoyer **null** ou bien une instance "par défaut" de **BestInsertion**.

Question 24. Ajoutez dans la classe **Planning** une méthode **planificationBestInsertion(Depot depot, Set<Client> clients)**. Cette méthode parcourt les tournées actuelles du planning et, pour chacune, fait appel à la méthode **calculerBestInsertion**. Puis, elle réalise effectivement l'insertion à l'endroit qui permet de faire le plus petit détour. Si un client n'a pu être inséré dans aucune tournée, alors on crée une nouvelle tournée.

Question 25. Ajoutez dans la classe **Routage** une méthode **public static void comparerMethodes(String fichierEntree)**. La méthode lit le fichier d'entrée, initialise toutes les routes du réseau routier (avec un appel à la méthode **initialiserRoutes()**), puis elle fait appel aux trois méthodes de planification que vous avez écrit. Enfin, le planning fourni par chaque algorithme d'optimisation est écrit dans un fichier différent. Le fichier sera nommé de manière automatique. Les trois fichiers de sortie pourront être mis dans des répertoires différents.

Question 26. Testez vos algorithmes de planification sur chacun des fichiers de test disponibles sur Moodle.

Comparez vos résultats. Sont-ils conformes à vos attentes ?

Question 27. Il est fort probable que les solutions obtenues puissent être améliorées. Proposez de nouveaux algorithmes de planification afin d'améliorer vos résultats.

References

[1] Delannoy, C., *Programmer en Java, Eyrolles, 2014*